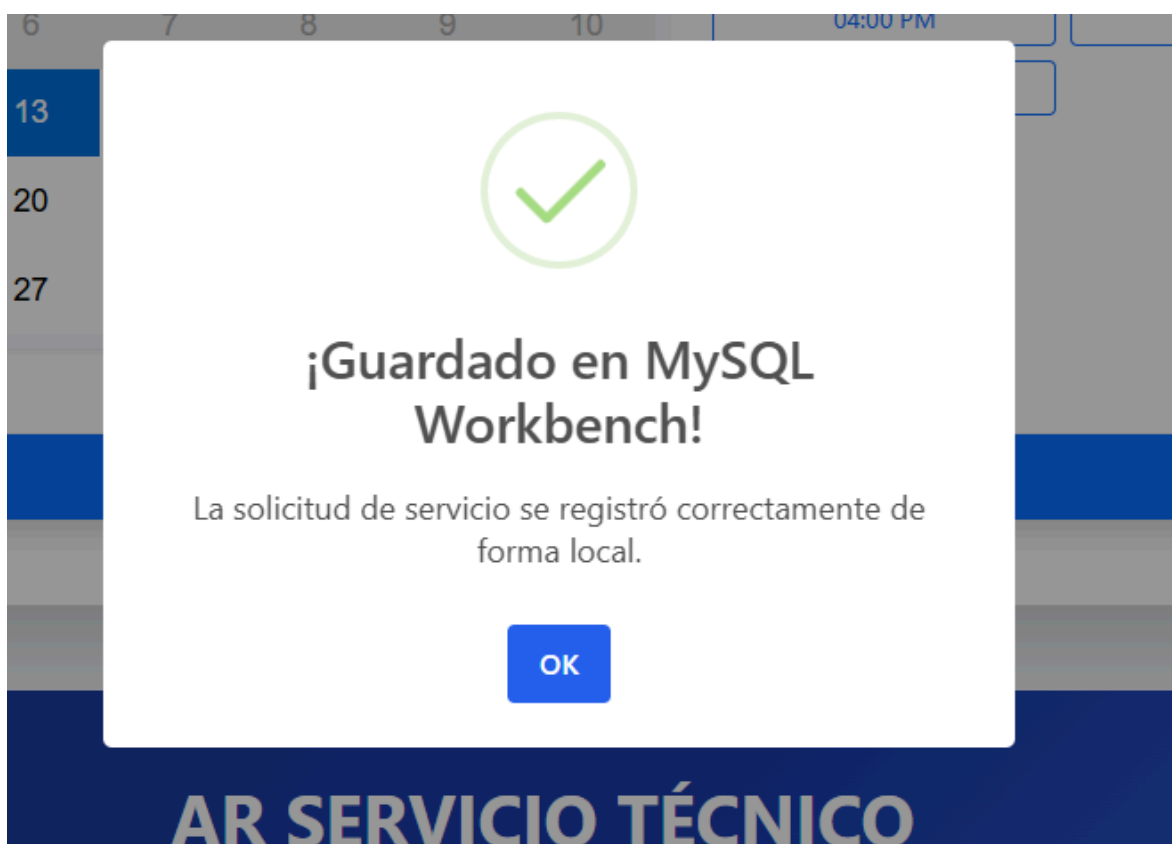


## Entorno de Desarrollo Local: MySQL Workbench

En la fase de desarrollo y pruebas internas, el módulo de solicitud opera de manera local para garantizar la integridad de los datos antes de su despliegue.

- **Flujo del Usuario:** Al completar el formulario de solicitud, el sistema valida los datos y confirma el registro exitoso mediante una interfaz visual intuitiva.
- **Persistencia de Datos:** La información se almacena en la base de datos local llamada serviat. Se ha verificado la correcta inserción mediante consultas SQL (`SELECT * FROM solicitud`), confirmando campos clave como `fecha_solicitud`, descripción y las llaves foráneas correspondientes.
- **Configuración del Servidor:** La conexión se gestiona mediante Node.js utilizando la librería `mysql2`, configurada en el host local (127.0.0.1) bajo el puerto 3306. Como lo vemos en las imágenes.



## Validar tipo de servicio

Por favor revisa lo detalles de tu solicitud antes de confirmar

### Resumen del servicio

Servicio:

Equipo:

**Cafetera (Industrial)**

Fecha y hora:

**a las**

Cliente:

**NICOLAS**

telefono:

**34565432**

Modelo:

Dirección:

**calle 100**

Descripción del problema:

**es nueva y necesito instalarla en un bar**

239

239 • `select * from solicitud;`

240

| Result Grid  |                 |                 |                    |                          |                    |                     |           |
|--------------|-----------------|-----------------|--------------------|--------------------------|--------------------|---------------------|-----------|
| Filter Rows: |                 | Edit:           |                    | Export/Import:           |                    | Wrap Cell Content:  |           |
| id_solicitud | fecha_solicitud | descripcion     | direccion_servicio | usuario_id_administrador | usuario_id_cliente | id_estado_solicitud | id_equipo |
| 37           | 2024-04-06      | Otro técnico    | Calle 37           | 3                        | 37                 | 4                   | 37        |
| 38           | 2024-04-07      | Tiempo          | Calle 38           | 3                        | 38                 | 4                   | 38        |
| 39           | 2024-04-08      | Rechazo         | Calle 39           | 3                        | 39                 | 4                   | 39        |
| 40           | 2024-04-09      | Cancelado       | Calle 40           | 3                        | 40                 | 4                   | 40        |
| 41           | 2026-05-13      | es nueva y n... | calle 100          | 1                        | 6                  | 1                   | 51        |
| HULL         | HULL            | HULL            | HULL               | HULL                     | HULL               | HULL                | HULL      |

solicitud 2 x

El código de la conexión con workbench:

```

server >  db.js > ...
1  import mysql from 'mysql2';
2
3  // Configuración de la conexión en workbench
4  const db = mysql.createConnection({
5    host: '127.0.0.1',
6    port: 3306,
7    user: 'root',
8    password: '',
9    database: 'serviat'
10 });
11
12
13 db.connect((err) => {
14   if (err) {
15     console.error('✗ Error al conectar a la base de datos:', err.message);
16     return;
17   }
18   console.log('✓ Conexión exitosa a MySQL (serviat)');
19 });
20
21
22 export default db;

```

login.jsx
solicitudFormulario.jsx
formulario\_solicitud
db.js
server.js
cliente.js
App.jsx
.env.local

 server.js > ...

```

1  import express from 'express';
2  import cors from 'cors';
3  import db from './server/db.js'; // Tu conexión a Workbench
4
5  const app = express();
6  const PORT = 3001;
7
8
9  app.use(express.json()); // permite leer el req.body de los formularios
10
11 app.use(cors({
12   origin: 'http://localhost:5173',
13   methods: ['GET', 'POST', 'PUT', 'DELETE'],
14   allowedHeaders: ['Content-Type', 'Authorization']
15 }));
16
17 app.get('/api/servicios', (req, res) => {
18   res.json({ mensaje: "CORS configurado exitosamente" });
19 });
20
21 //para la solicitud o fomulario
22 app.post('/api/solicitud', (req, res) => {
23   const {
24     nombre_equipo,
25     modelo_equipo,

```

PROBLEMAS
SALIDA
CONSOLA DE DEPURACIÓN
TERMINAL
PUERTOS

 node + -   ...

```

PS C:\Users\Windows\Downloads\serviat> npm install mysql2
added 9 packages, and audited 270 packages in 3s

70 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Windows\Downloads\serviat> node server.js
✓ Servidor con CORS listo en el puerto 3001
✓ Conexión exitosa a MySQL (serviat)

```

Lín. 8, Col. 1 (2 seleccionada)
Espacios: 2
UTF-8
CRLF
() JavaScript
Sign In
Go

## Entorno de Producción en la Nube: Supabase

Para la escalabilidad y disponibilidad del sistema, se implementó una integración con **Supabase (PostgreSQL)**, permitiendo el acceso remoto a la información de los servicios.

- **Gestión de Relaciones:** El sistema no solo guarda la solicitud, sino que realiza una inserción escalonada:
  1. **Validación de Usuario:** Recupera la sesión activa desde el localStorage.
  2. **Resolución de IDs:** Busca dinámicamente los identificadores de la categoría del equipo y el estado inicial (por defecto "Pendiente").
  3. **Inserción de Equipo:** Crea el registro en la tabla equipo y recupera el ID generado.
  4. **Finalización de Solicitud:** Vincula todos los datos previos para crear el registro final en la tabla solicitud.

```
21
22 select * from equipo;
```

| Results Explain Chart Export ▾ |               |                 |               |              |
|--------------------------------|---------------|-----------------|---------------|--------------|
| id_equipo                      | nombre_equipo | marca_equipo    | modelo_equipo | id_categoria |
| 43                             | horno         | No especificada | N/A           | 1            |
| 44                             | horno         | No especificada | N/A           | 1            |
| 45                             | horno         | No especificada | N/A           | 1            |
| 46                             | horno         | No especificada | N/A           | 1            |
| 47                             | horno         | No especificada | N/A           | 1            |
| 48                             | horno         | No especificada | N/A           | 1            |

9 rows (Limited to only 100 rows)

```

21
22 select * from solicitud;

```

| id_solicitud | fecha_solicit | descripcion | direccion_ser | usuario_id_ad | usuario_id_cl | id_estado_sol | id_equipo |
|--------------|---------------|-------------|---------------|---------------|---------------|---------------|-----------|
| 5            | 2026-05-13    | no prende   | calle 100     | 1             | 6             | 1             | 48        |

Código de la conexión:

```

try {
    const userGuardado = JSON.parse(localStorage.getItem("user"));
    if (!userGuardado) {
        throw new Error("Debes iniciar sesión para agendar un
servicio.");
    }

    // Buscar id de categoría dinámicamente
    const { data: catData, error: catError } = await cliente
        .from('categoria_equipo')
        .select('id_categoria_equipo')
        .eq('nombre_categoria', datosValidar.categoria)
        .single();

    if (catError || !catData) {
        throw new Error("Categoría no encontrada: " +
datosValidar.categoria);
    }

    // Buscar id del estado dinámicamente
    const { data: estadoData, error: estadoError } = await cliente
        .from('estado') // Asegúrate de poner el nombre exacto de tu
tabla
        .select('id_estado')
        .eq('nombre_estado', 'Pendiente')
        .single();

    if (estadoError || !estadoData) {
        throw new Error("Estado 'Pendiente' no encontrado en la base
de datos.");
    }

    const idEstadoPendiente = estadoData.id_estado;

```

```

const idCategoria = catData.id_categoria_equipo;
const idAdministrador = 1;

// Insertar equipo
const { data: equipoInsertado, error: equipoError } = await
cliente
    .from('equipo')
    .insert([
        {
            nombre_equipo: datosValidar.equipo,
            marca_equipo: "No especificada",
            modelo_equipo: datosValidar.modelo || "N/A",
            id_categoria_equipo: idCategoria
        }
    ])
    .select();

if (equipoError) throw equipoError;

const idDelNuevoEquipo = equipoInsertado[0].id_equipo;

// Insertar solicitud
const { error: errorSolicitud } = await cliente
    .from('solicitud')
    .insert([
        {
            fecha_solicitud:
fechaSeleccionada.toISOString().split('T')[0],
            descripcion: datosValidar.descripcion,
            direccion_servicio: datosValidar.direccion,
            usuario_id_administrador: idAdministrador,
            usuario_id_cliente: userGuardado.id,
            id_estado_solicitud: idEstadoPendiente,
            id_equipo: idDelNuevoEquipo
        }
    ]);

if (errorSolicitud) throw errorSolicitud;

await Swal.fire({
    icon: "success",
    title: "¡Solicitud Exitosa!",
    text: `Tu servicio ha sido agendado con ${tecnicoNombre}.`,
    confirmButtonColor: '#2563eb'
});

```